МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний аерокосмічний університет
«Харківський авіаційний інститут»

Факультет систем управління літальних апаратів
Кафедра систем управління літальних апаратів

Лабораторна робота № 9

з дисципліни «Алгоритмізація та програмування»
на тему «Робота з рядками на С ++»

ХАІ.301. G3. 319а. 25 ЛР

Виконав студент гр. _____319а_____

_____Андрій НОЗДРЯКОВ___
(підпис, дата) (П.І.Б.)

Перевірив

_____ асистент Євген ПЯВКА
(підпис, дата) (П.І.Б.)

2026

МЕТА РОБОТИ

Вивчити теоретичний матеріал з основ роботи з низькорівневими рядками на C++ і документацію до класу string, а також алгоритми пошуку в рядку, а також реалізувати обробку рядків на C++ в середовищі QtCreator.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1.

A) Вивчити по документації метод стандартного класу string відповідно до варіанту.
B) Визначити функцію, що виконує ті ж дії, що і вивчений метод класу string. Вихідний рядок передати першим параметром (масив символів). Для реалізації методу не використовувати функції обробки рядків зі стандартних бібліотек.
C) Викликати свій метод і метод string аналогічно прикладам коду, наведеними в дод.А. *Перед викликом ввести з консолі один рядок і зберегти в масиві символів і змінній типу string.

Варіант 28: size_t rfind (const string& str, size_t pos = npos) const;

Завдання 2.

A) Описати функцію, що обробляє рядок відповідно до завдання. Для реалізації можна використовувати функції обробки рядків зі стандартних бібліотек
B) Описати функцію, яка перевіряє, чи задовольняє рядок умовам завдання.
C) Створити вихідний текстовий файл, що містить не менше 10 різних рядків.
D) Використовуючи функції 2.A і 2.B, обробити рядок / * текстовий файл рядок за рядком. Додаткові дані ввести з консолі.
E) Отриманий результат записати у вихідний файл.

String39: Дано рядок, що містить принаймні один символ пробілу. Вивести підстроку, розташовану між першим і другим пробілом початкового рядка. Якщо рядок містить тільки один пробіл, то вивести порожній рядок.

Завдання 3.

Завдання 1-2 реалізувати окремими функціями без параметрів, у функції main() організувати меню для багаторазового виконання завдань. Структурувати проєкт програми: винести заголовки і реалізацію функцій в окремі .h та .cpp файли.

Завдання 4. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок за допомогою наступних промптів:

1) «Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»

2) «Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку»

3) «Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань»

Проаналізуйте задані питання, коментарі і оцінки, надані ШІ. Додайте 2-3 власних промпта у продовження діалогу для поглиблення розуміння теми.

ВИКОНАННЯ РОБОТИ

Завдання 1.

Вирішення задачі Варіант 28

Вхідні дані (ім’я, опис, тип, обмеження):

cstr - Основний рядок, у якому здійснюється пошук, char[100], До 99 символів + нуль-термінатор.

Csubstr - Підрядок, позицію якого треба знайти, char[100], До 99 символів.

Вихідні дані (ім’я, опис, тип):

pos - Результат стандартного методу string::rfind, size_t / int;

my_pos - Результат вашої власної функції my_rfind, int;

Алгоритм вирішення Варіант 28 показано нижче/на рис. 1

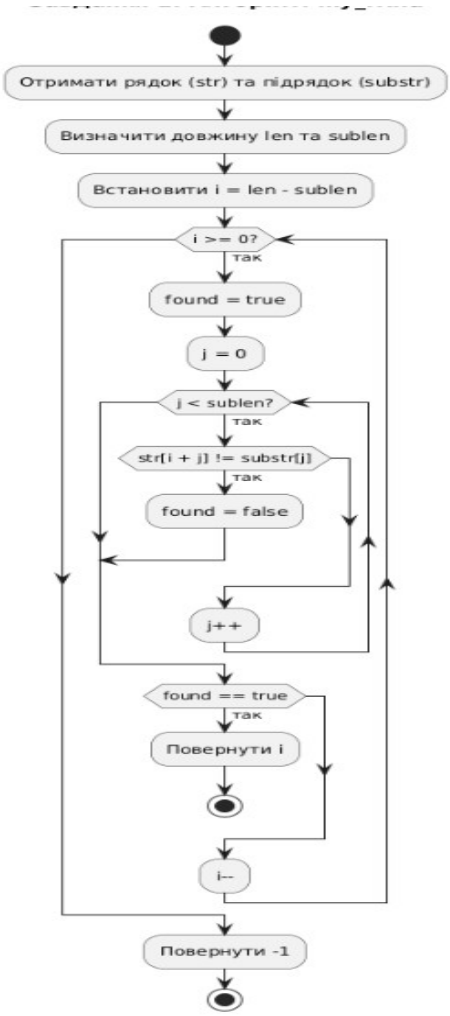

Рисунок 1 – Алгоритм вирішення Варіант 28

Лістинг коду вирішення задачі розділ і номер задач(і) наведено в дод. А (стор. х).

Екран роботи програми показаний на рис. Б.1.

Завдання 2.

Вирішення задачі String 39

Вхідні дані (ім’я, опис, тип, обмеження):

input.txt - Текстовий файл із вхідними рядками., ifstream , Файл має існувати в директорії з програмою.

line - Окремий рядок, зчитаний з файлу, string , Довжина обмежена об'ємом оперативної пам'яті. .

Вихідні дані (ім’я, опис, тип):

output.txt - Текстовий файл, куди записуються результати обробки, ofstream;

5

substring - Текст, що знаходився між першим та другим пробілами, string;

Алгоритм вирішення String 39 показано нижче/на рис. 2

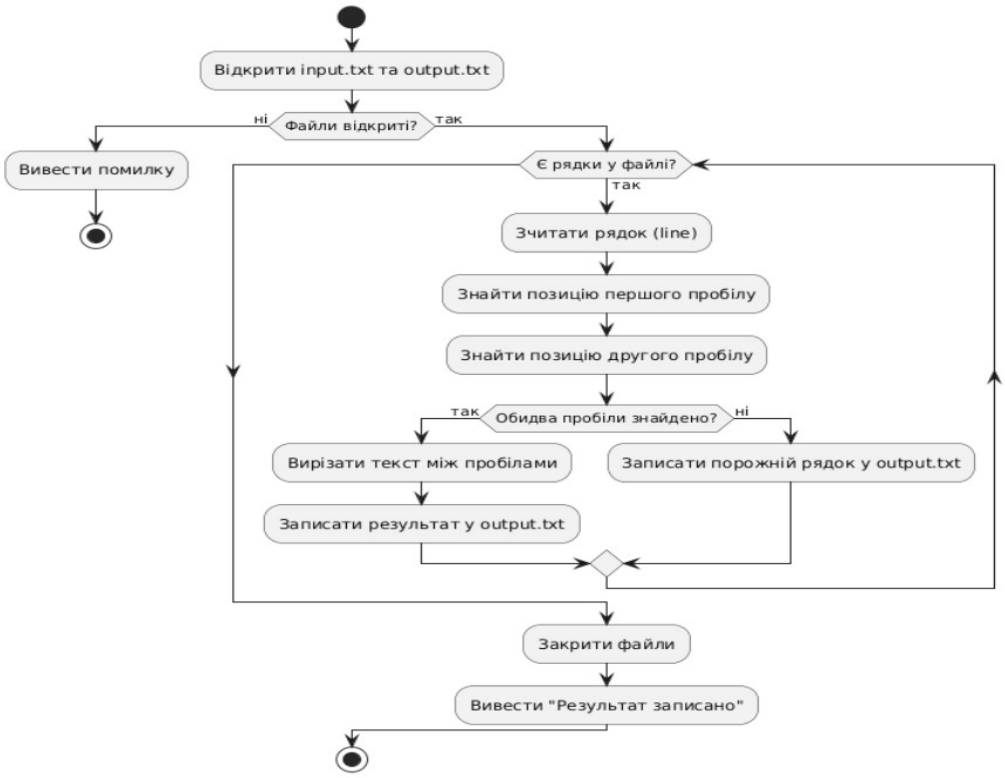

Рисунок 2 – Алгоритм вирішення String 39

Лістинг коду вирішення задачі розділ і номер задач(і) наведено в дод. А (стор. х).

Екран роботи програми показаний на рис. Б.2.

Завдання 3.

Вирішення задачі Menu

Вхідні дані (ім’я, опис, тип, обмеження):

choice - Команда, яку вводить користувач для вибору дії, int , Допустимі значення: 0, 1, 2.

Вихідні дані (ім’я, опис, тип):

Текстове повідомлення - Відображення списку доступних команд у консолі, cout ;

Повідомлення про помилку - Виводиться, якщо введено число, якого немає в списку, cout ;

Алгоритм вирішення Menu показано нижче/на рис. 3

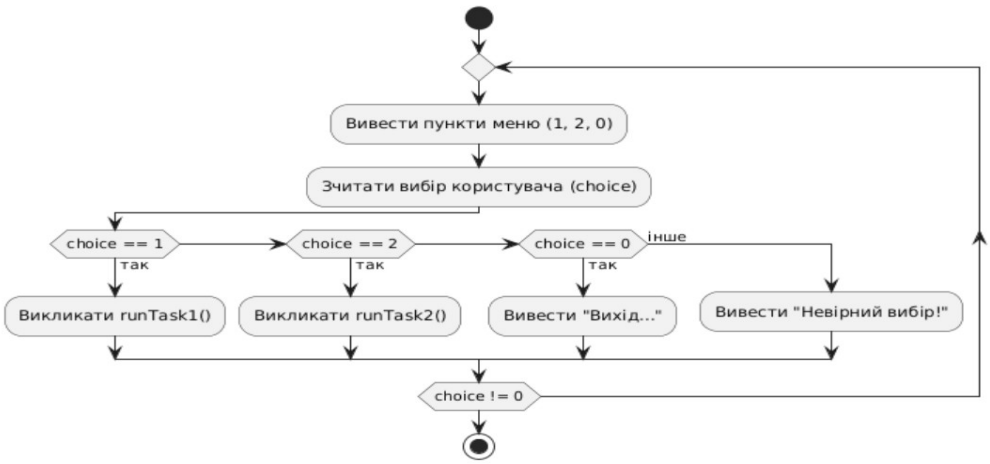

Рисунок 3 – Алгоритм вирішення Menu

Лістинг коду вирішення задачі розділ і номер задач(і) наведено в дод. А (стор. х).

Екран роботи програми показаний на рис. Б.1.

ВИСНОВКИ

На лабораторній роботі було вивчено теоретичний матеріал з основ роботи з низькорівневими рядками на C++ і документацію до класу string, а також алгоритми пошуку в рядку, а також реалізувати обробку рядків на C++ і закріплено все це на практиці.Труднощів з не виникало.

ДОДАТОК А

Лістинг коду програми

```
   #include <iostream>
#include <fstream>
#include <string>

using namespace std;

// ===================== ЗАВДАННЯ 1 =====================

// власна реалізація rfind
int my_rfind(const char* str, const char* substr) {
    int len = 0, sublen = 0;

    while (str[len] != '\0') len++;
    while (substr[sublen] != '\0') sublen++;

    for (int i = len - sublen; i >= 0; i--) {
        bool found = true;

        for (int j = 0; j < sublen; j++) {
            if (str[i + j] != substr[j]) {
                found = false;
                break;
            }
        }

        if (found) return i;
    }

    return -1;
}

void runTask1() {
    char cstr[100];
    char csubstr[100];

    cin.ignore();
    cout << "Введіть рядок: ";
    cin.getline(cstr, 100);

    cout << "Введіть підрядок: ";
    cin.getline(csubstr, 100);

    string str = cstr;
    string substr = csubstr;

    size_t pos = str.rfind(substr);
```

```
    int my_pos = my_rfind(cstr, csubstr);

    cout << "\nstring::rfind: ";
    if (pos != string::npos)
        cout << pos << endl;
    else
        cout << "не знайдено\n";

    cout << "my_rfind: ";
    if (my_pos != -1)
        cout << my_pos << endl;
    else
        cout << "не знайдено\n";
}

// ===================== ЗАВДАННЯ 2 =====================

// A. обробка рядка
string getSubstringBetweenSpaces(const string& str) {
    size_t first = str.find(' ');
    if (first == string::npos) return "";

    size_t second = str.find(' ', first + 1);
    if (second == string::npos) return "";

    return str.substr(first + 1, second - first - 1);
}

// B. перевірка
bool isValid(const string& str) {
    size_t first = str.find(' ');
    if (first == string::npos) return false;

    size_t second = str.find(' ', first + 1);
    return second != string::npos;
}

void runTask2() {
    ifstream fin("input.txt");
    ofstream fout("output.txt");

    if (!fin.is_open()) {
        cout << "Помилка відкриття input.txt\n";
        return;
    }

    string line;

    while (getline(fin, line)) {
        if (isValid(line))
            fout << getSubstringBetweenSpaces(line) << endl;
```

```
        else
            fout << "" << endl;
    }

    fin.close();
    fout.close();

    cout << "Результат записано у output.txt\n";
}

// ===================== МЕНЮ =====================

int main() {
    int choice;

    do {
        cout << "\n===== МЕНЮ =====\n";
        cout << "1. Завдання 1 (rfind)\n";
        cout << "2. Завдання 2 (робота з рядками і файлом)\n";
        cout << "0. Вихід\n";
        cout << "Ваш вибір: ";
        cin >> choice;

        switch (choice) {
            case 1:
                runTask1();
                break;
            case 2:
                runTask2();
                break;
            case 0:
                cout << "Вихід...\n";
                break;
            default:
                cout << "Невірний вибір!\n";
        }

    } while (choice != 0);

    return 0;
}
```

ДОДАТОК Б

Скрін-шоти вікна виконання програми

```
===== МЕНЮ =====
1. Завдання 1 (rfind)
2. Завдання 2 (робота з рядками і файлом)
0. Вихід
Ваш вибір: 1
Введіть рядок: Hello world, this is world!
Введіть підрядок: world

string::rfind: 22
my_rfind: 22

===== МЕНЮ =====
1. Завдання 1 (rfind)
2. Завдання 2 (робота з рядками і файлом)
0. Вихід
Ваш вибір: 0
Вихід...
```

Рисунок Б.1 – Екран виконання програми для вирішення завдання Варіант 28, Menu;

```
===== МЕНЮ =====
1. Завдання 1 (rfind)
2. Завдання 2 (робота з рядками і файлом)
0. Вихід
Ваш вибір: 2

Результат записано у output.txt
```

input.txt - Блокнот

Файл Правка Формат Вид Справка

```
Ivan Petrov 25
Mary Jane Smith
hello world
John Ronald Reuel Tolkien
C++ is awesome
one two three four
```

output.txt - Блокнот

Файл Правка Формат Вид Справка

```
Petrov
Jane

Ronald
is
```

Рисунок Б.2 – Екран виконання програми для вирішення завдання String 39

ДОДАТОК В

Діалог з ШІ для самоаналізу

Студент:
Надіслав код
Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції

Чат GPT:
Добре, уявімо, що ти на захисті ? Я — викладач. Питання будуть не просто на запам'ятовування, а на розуміння логіки твого коду.

ТЕСТОВІ ПИТАННЯ (обери 1 правильну відповідь)

1. Що поверне функція my_rfind, якщо підрядок не знайдено?
A. 0
B. -1
C. string::npos
D. Останній індекс рядка

---

2. Яка головна відмінність між my_rfind і string::rfind?
A. my_rfind працює швидше
B. string::rfind шукає з початку
C. my_rfind працює з char*, а rfind — з string
D. Вони працюють абсолютно однаково

---

3. Що станеться, якщо substr довший за str у my_rfind?
A. Програма завершиться аварійно
B. Повернеться 0
C. Повернеться -1
D. Буде нескінченний цикл

13

4. Що робить ця стрічка?

return str.substr(first + 1, second - first - 1);

A. Видаляє пробіли

B. Повертає підрядок між двома пробілами

C. Повертає весь рядок без першого слова

D. Повертає останнє слово

---

5. Навіщо потрібна функція isValid()?

A. Для оптимізації швидкодії

B. Для перевірки довжини рядка

C. Щоб перевірити, чи є хоча б два пробіли

D. Для очищення рядка

Студент:

ТЕСТИ (Правильні варіанти)B. -1(У твоїй реалізації int my_rfind повертає -1, оскільки це класичний спосіб позначити "не знайдено" для цілих чисел).C. my_rfind працює з char*, а rfind — з string(Твоя функція приймає вказівники на символи, тоді як метод класу std::string працює з об'єктами цього класу).C. Повернеться -1(У циклі for (int i = len - sublen; ...) значення len - sublen буде від'ємним, цикл просто не почнеться, і функція дійде до return -1).B. Повертає підрядок між двома пробілами(Формула second - first - 1 обчислює точну кількість символів між позиціями пробілів).C. Щоб перевірити, чи є хоча б два пробіли(Це запобіжник: якщо пробілів менше ніж два, метод substr може отримати некоректні індекси).? ВІДКРИТІ ПИТАННЯ1. Алгоритм my_rfindАлгоритм працює методом "ковзного вікна", але рухається справа наліво.Чому з кінця? Бо задача rfind (reverse find) — знайти останнє входження. Якби ми йшли з початку, нам би довелося пройти весь рядок і постійно оновлювати змінну останньої знайденої позиції. Пошук з кінця дозволяє зупинитися одразу, як тільки знайдено перший збіг.Складність: У найгіршому випадку (наприклад, пошук "aaaa" в "aaaaaaaa") це $O(N \cdot M)$, де N — довжина рядка, а M — довжина підрядка.2. Потенційні помилки з cin.ignore()Головна проблема — непередбачуваність стану буфера.Якщо користувач введе число в меню і натисне Enter, cin >> choice залишить \n у

буфері. Тоді ignore() спрацює ідеально.Але якщо ти викличеш ignore() там, де буфер уже порожній (наприклад, всередині функції, яка викликається повторно), програма "зависне", чекаючи, поки ти натиснеш будь-яку клавішу, і просто "проковтне" перший символ твого наступного введення.3. Поведінка з "Hello world example text"У файл запишеться слово world.Чому? Програма шукає перший пробіл (після "Hello") і другий пробіл (після "world"). Функція substr виріже все, що знаходиться між ними. Третій пробіл і слово "example" будуть просто ігноруватися, бо логіка зупиняється на другому знайденому пробілі.4. Зміна на "між першим і останнім"Для цього потрібно змінити пошук другого пробілу з find на rfind:C++size_t first = str.find(' '); size_t last = str.rfind(' '); // Шукаємо з кінця if (first = string::npos last = string::npos first = last) { return str.substr(first + 1, last - first - 1); } 5. Чому string краще за char[]?З точки зору інженерії:Безпека пам'яті : string автоматично керує пам'яттю.Не потрібно хвилюватися про переповнення буфера , якщо вхідний рядок виявиться довшим за 100 символів.Гнучкість: string динамічно розширюється і не треба заздалегідь знати максимальну довжину рядка у файлі.Готові методи: Наявність вбудованих методів як .find(), .substr(), .append() робить код читабельним і зменшує кількість помилок "off-by-one" (помилка на одиницю при підрахунку індексів).

Чат GPT:

ТЕСТИ — перевірка

✅1. B (-1) — правильно

Чітке розуміння контракту функції

✅*2. C (char vs string)** — правильно

Це ключова різниця на рівні типів і абстракцій

3. C (-1) — *правильна відповідь*

- Так, цикл не почнеться
- Але причина — не просто “від’ємне значення”, а те, що:

for (int i = len - sublen; i >= 0; i--)

якщо len - sublen < 0, умова i >= 0 одразу false

В цілому — зараховано

15

✅4. B (підрядок між пробілами) — правильно

Гарне розуміння індексів

✅5. C (перевірка на 2 пробіли) — правильно

І правильно пояснив як “запобіжник”

Результат тестів: 5/5 (з мінімальним зауваженням)

ВІДКРИТІ ПИТАННЯ — оцінка

1. Алгоритм my_rfind

Дуже сильна відповідь

Ти:

- правильно назвав підхід (“ковзне вікно”)
- пояснив, чому з кінця ефективніше
- дав складність O(N·M)

---

2. cin.ignore()

✅Відмінно

Ти зловив головну проблему:

- буфер вводу
- зависання
- “проковтування” символів

Це дуже часта помилка — плюс тобі

---

3. Поведінка рядка

✅Ідеально

Чітко:

- що виведе → world

16

- чому → перший і другий пробіл

---

4. Модифікація через rfind

✅Абсолютно правильно

Це саме те рішення, яке очікує викладач

---

5. string vs char[]

✅Сильна інженерна відповідь

Ти згадав:

- безпеку пам’яті
- динамічність
- зручні методи

ЗАГАЛЬНА ОЦІНКА ЯК НА ЗАХИСТІ

Рівень відповіді: 10 / 10

ВІДКРИТІ ПИТАННЯ (відповідай своїми словами)

1.

Поясни, як працює алгоритм у my_rfind.
Чому пошук іде з кінця, і яка його складність?

2.

Які потенційні помилки можуть виникнути при використанні:

cin.ignore();
у твоєму коді?

3.

Як зміниться поведінка програми, якщо у рядку буде:

17
Hello world example text
Що саме запишеться у файл і чому?

4.
Як би ти змінив getSubstringBetweenSpaces, щоб вона повертала текст між першим і останнім пробілом, а не першим і другим?

5.
Чому в другому завданні краще використовувати string, ніж char[]? Наведи 2–3 аргументи з точки зору інженерії.

Студент:
Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку

Чат GPT:

ТЕСТОВІ ПИТАННЯ

1. (-1)

- Правильність: 100%
- Повнота: достатня
- Ймовірність ШІ: низька (просте питання)

Оцінка: 5/5

---

*2. (char vs string)**

- Правильність: 100%
- Повнота: достатня
- Ймовірність ШІ: низька

Оцінка: 5/5

---

3. (довший підрядок → -1)

- Правильність: 100%
- Повнота: трохи поверхнево пояснено
- Ймовірність ШІ: середня (формулювання “типове”)

штраф НЕ застосовую (не критично)

Оцінка: 4/5

---

4. (substr між пробілами)

- Правильність: 100%
- Повнота: хороша
- Ймовірність ШІ: низька

Оцінка: 5/5

---

5. (isValid)

- Правильність: 100%
- Повнота: достатня
- Ймовірність ШІ: низька

Оцінка: 5/5

? Середнє за тести:

(5+5+4+5+5)/5=4.8(5 + 5 + 4 + 5 + 5) / 5 = 4.8(5+5+4+5+5)/5=4.8

ВІДКРИТІ ПИТАННЯ

---

1. Алгоритм my_rfind

- Правильність: повна
- Повнота: висока (є складність, пояснення)
- ?Ймовірність ШІ:низька

19
Оцінка: 5/5

---

2. cin.ignore()

- Правильність: повна
- Повнота: висока
- Ймовірність ШІ: низька

Оцінка: 5/5

3. Поведінка рядка
Правильність: повна
Повнота: достатня (пояснення логіки substring між першим і другим пробілом)
Ймовірність ШІ: низька

Оцінка: 5/5

---

4. Модифікація через rfind
Правильність: ідеальна
Повнота: висока (наведено код)
Ймовірність ШІ: низька

Оцінка: 5/5

---

5. string vs char[]
Правильність: повна
Повнота: дуже висока (кілька інженерних аргументів)
Ймовірність ШІ: низька

Оцінка: 5/5

Середня оцінка за відкриті питання

(5+5+5+5+5)/5=5

20

Чат GPT:

1. Аналіз наявного коду

У тебе реалізовано:

Завдання 1

- Власна реалізація пошуку підрядка з кінця (my_rfind)
- Порівняння з std::string::rfind

Завдання 2

- Робота з файлами (ifstream, ofstream)
- Обробка рядків:
  - пошук пробілів
  - виділення підрядка між ними
  - перевірка валідності рядка

2. Що є в теорії, але НЕ реалізовано в коді

Нижче — важливі концепти з теми “рядки в C++”, які логічно мають бути у звіті, але у твоєму коді не відпрацьовані.

2.1. Перевантажені методи find() / rfind() з позицією старту

Теорія

У std::string можна шукати з конкретної позиції:

size_t pos = str.find("abc", 5);

Приклад

```
#include <iostream>
#include <string>
using namespace std;
int main() {
   string s = "abc abc abc";
   size_t first = s.find("abc");
   size_t second = s.find("abc", first + 1);
   cout << first << " " << second << endl;
}
```

Чому це важливо

У твоєму коді це не використано, але це:

- дозволяє уникати ручних циклів
- спрощує алгоритми пошуку

2.2. substr() як універсальний механізм розбиття рядків

Теорія

substr(pos, len) дозволяє будувати парсери.

Приклад (розбиття на слова)

```
string s = "one two three";
size_t start = 0;
size_t space = s.find(' ');
while (space != string::npos) {
   cout << s.substr(start, space - start) << endl;
   start = space + 1;
   space = s.find(' ', start);
}
cout << s.substr(start) << endl;
```

Що це дає

- повна обробка будь-якої кількості слів
- заміна ручних перевірок first/second

У тебе зараз обробляється тільки 2 пробіли

2.3. Обробка string::npos

Теорія

npos — це спеціальне значення “не знайдено”.

Приклад

```
string s = "hello";
```

```
if (s.find("x") == string::npos) {
   cout << "not found";
}
```

Важливо

У твоєму коді це використано частково, але не пояснено у логіці звіту

2.4. Безпечна робота з буферами (getline vs cin >>)

Теорія

Різниця:

Метод	Поведінка
cin >>	зупинка на пробілі
getline	читає весь рядок

Приклад

```
string name;
cin >> name;        // "Ivan Petrov" → тільки Ivan
getline(cin, name); // "Ivan Petrov" → повністю
```

У твоєму коді

✔ getline використано правильно

✕ але теоретичне пояснення відсутнє

2.5. Робота з потоками файлів (режими відкриття)

Теорія

Файли можна відкривати в різних режимах:

```
ofstream fout("file.txt", ios::app); // додавання
ofstream fout("file.txt", ios::trunc); // перезапис
```

Приклад

```
ofstream f("log.txt", ios::app);
f << "New line\n";
```

23

У твоєму коді

- використано стандартний режим
- але режими не пояснені і не варіюються

2.6. Захист від переповнення в C-рядках

Теорія

char[] небезпечні через фіксований розмір.

Приклад проблеми:

```
char c[5];
cin >> c; // "hello world" → переповнення
```

Безпечний варіант:

```
cin.getline(c, 5);
```

У твоєму коді

✔ використано getline

✕ але тема безпеки не розкрита в теорії

3. Висновок по звіту

Сильні сторони

- є власна реалізація алгоритму
- є порівняння зі STL
- є робота з файлами
- є базова обробка рядків

Що НЕ повністю покрито теорією

- багатократний пошук через find(pos)
- повноцінний парсинг рядків (не лише 2 пробіли)
- робота з npos як концепція

24

- режими відкриття файлів
- глибша безпека C-рядків
- порівняння getline vs cin >> як теорія

Підсумок як викладач

Практика: добра (80–90%)
Теорія: частково розкрита (60–70%)
Звіт: потребує доповнення теоретичного блоку